CKG-LLM: LLM-Assisted Detection of Smart Contract Access Control Vulnerabilities Based on Knowledge Graphs

Xiaoqi Li
Hainan University
Haikou, China
csxqli@ieee.org

Hailu Kuang*
Hainan University
Haikou, China
hailukuang@hainanu.edu.cn

Wenkai Li
Hainan University
Haikou, China
cswkli@hainanu.edu.cn

Zongwei Li
Hainan University
Haikou, China
lizw1017@hainanu.edu.cn

Shipeng Ye
Hainan University
Haikou, China
shipengye@hainanu.edu.cn

Abstract

Traditional approaches for smart contract analysis often rely on intermediate representations such as abstract syntax trees, control-flow graphs, or static single assignment form. However, these methods face limitations in capturing both semantic structures and control logic. Knowledge graphs, by contrast, offer a structured representation of entities and relations, enabling richer intermediate abstractions of contract code and supporting the use of graph query languages to identify rule-violating elements. This paper presents CKG-LLM, a framework for detecting access-control vulnerabilities in smart contracts. Leveraging the reasoning and code generation capabilities of large language models, CKG-LLM translates natural-language vulnerability patterns into executable queries over contract knowledge graphs to automatically locate vulnerable code elements. Experimental evaluation demonstrates that CKG-LLM achieves superior performance in detecting access-control vulnerabilities compared to existing tools. Finally, we discuss potential extensions of CKG-LLM as part of future research directions.

CCS Concepts

• **Security and privacy → Software security engineering**.

Keywords

Blockchain Security, LLM, Knowledge Graph, Access Control

ACM Reference Format:
Xiaoqi Li, Hailu Kuang, Wenkai Li, Zongwei Li, and Shipeng Ye. 2026. CKG-LLM: LLM-Assisted Detection of Smart Contract Access Control Vulnerabilities Based on Knowledge Graphs. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE-NIER '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3786582.3786817

*The corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE-NIER '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2425-1/26/04
https://doi.org/10.1145/3786582.3786817

1 Introduction

Smart contracts are blockchain-based programs that automatically execute predefined rules [42] [48] [15] [18]. Knowledge graphs (KGs) [10] can represent real-world entities and relations, and also serve as an intermediate representation (IR) of the semantic structure and execution logic of smart contracts. Hu et al. [12] show that a contract knowledge graph (CKG) can be constructed by parsing abstract syntax trees (ASTs) [31] and using graph query languages (GQL) to identify vulnerabilities. However, CKGs built from ASTs often provide shallow semantic representations and overlook deeper syntactic and structural information. In addition, the difficulty of designing precise SPARQL queries to locate entities or relations limits the promotion of KG-based vulnerability detection [34].

To address these challenges, we propose CKG-LLM, a LLM-assisted framework for detecting access control vulnerabilities based on smart contract knowledge graphs. The framework exploits Slither's IR [6] to extract diverse instances, containment relations, control-flow structures, and invocation dependencies, thereby constructing a semantically enriched CKG. Furthermore, the framework leverages the code generation and reasoning capabilities of large language models (LLMs) [1] to translate natural-language (NL) vulnerability patterns into graph queries, thereby eliminating the need for rule-based parsing. To enhance query accuracy and robustness, we further introduce RLAF, a domain-adaptive reinforcement learning approach that incorporates feedback from the language model agent, enabling iterative refinement of query generation.

The main contributions of this paper are summarized as follows:

- We propose CKG-LLM, to the best of our knowledge, the first framework that leverages LLMs to translate natural language descriptions into graph queries over smart contract knowledge graphs, enabling access control vulnerability detection.
- We develop a contract knowledge graph construction pipeline based on Slither's intermediate representation, and introduce a reinforcement learning from small language model agent feedback (RLAF) to guide LLMs in generating high-quality queries.
- Through evaluation, CKG-LLM achieved an average accuracy of 73.1% and an F1 score of 74.9%, demonstrating higher detection efficiency compared to existing access-control analysis tools.

In future work, we plan to extend CKG-LLM to cover additional classes of vulnerabilities and refine its pipeline by integrating retrieval-augmented generation (RAG) [13] for SPARQL query generation, thereby improving both scalability and accuracy.

2 Background

Access control has risen to the top of the OWASP rankings, reflecting its growing severity [32] [14] [2]. It is especially critical in smart contracts where autonomous and immutable code makes authorization flaws capable of causing irreversible asset loss [16] [17] [19] [33]. Consequently, precise detection of access control vulnerabilities is essential [8] [37] [45]. Recent work demonstrates that LLMs can strengthen smart contract security pipelines by complementing static analysis [41] [20], symbolic execution [3] [46], and fuzz testing [38] [47]. Meanwhile, KG-based analysis organizes contract syntax and logic into semantically enriched graph structures that capture ontological and instance-level relationships [12]. As LLMs continue to excel in natural-language-to-graph-query translation (NL2GQL) [49] [21], users can express security requirements or vulnerability patterns in natural language, enabling models to generate queries over KG-encoded contracts. This combination of LLM reasoning and KG representations enhances domain awareness, overcomes traditional reasoning limitations, and supports scalable detection of identity and authorization-related vulnerabilities.

3 CKG-LLM

CKG-LLM is a graph-query-driven framework for smart contract vulnerability detection. It employs domain-enhanced fine-tuning and reinforcement learning (RL) [9] to guide LLMs in generating GQL queries, thereby enabling the identification of vulnerabilities within contract knowledge graphs. The overall workflow, shown in Figure 2, comprises four stages: (1) contract knowledge graph construction and prompt design; (2) supervised fine-tuning for NL2GQL; (3) DPO-based reinforcement learning from small language model agent feedback (RLAF); and (4) framework deployment and application. Each stage is described in the following sections.

3.1 CKG Building

The CKG employs a two-layered design, comprising the ontology and the instance layer. The ontology layer formalizes the abstract syntax of Solidity, serving as a domain schema, while the instance layer represents contract structures and their interrelations.

3.1.1 Ontology. The ontology layer construction is informed by the Solidity language grammar [39] and the ANTLR grammar specification [40]. Formally, we define the ontology as:

$$\mathcal{O} = (C, A, P),$$

where C denotes the set of ontology classes, A the set of class attributes, and $P = P_{\text{obj}} \cup P_{\text{data}}$ the set of properties, with $P_{\text{obj}} \subseteq C \times P \times C$ representing object properties that connect classes, and $P_{\text{data}} \subseteq C \times P \times A$ representing datatype properties that link classes to their attributes. As shown in Figure 1, ontology class nodes encompass syntactic constructs. These classes may also form inheritance-like relationships (e.g., Interface as a subclass of Contract). Class attribute nodes define the data types (e.g., the Identifier data type is string). Properties between nodes are further categorised as `has` or `is`, depending on their cardinality (e.g., Contract `hasStateVar` StateVar). We also introduce auxiliary nodes and edges to capture program logic and control flow (e.g., `hasNext` encodes execution order), facilitate efficient querying (e.g., `indexIs` specifies the position of statements or struct members), or improve interpretability to LLMs (e.g., `entryPointIs` links a function to its entry statement).

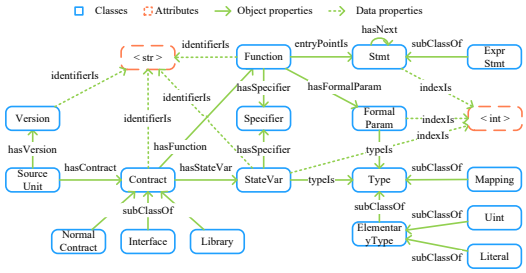


Figure 1: A simplified example of the CKG ontology layer.

3.1.2 Instance. The instance layer is constructed using Slither [6], which functions not only as a static analysis tool but also as a contract parser. Formally, the instance layer is represented as:

$$\mathcal{I} = (I, L, P),$$

where I denotes the set of instance nodes, L the set of literal values associated with specific instances, and $P = P^I_{\text{obj}} \cup P^I_{\text{data}}$ the set of instantiated properties, with $P^I_{\text{obj}} \subseteq I \times P \times I$ denoting object property assertions that connect instances, and $P^I_{\text{data}} \subseteq I \times P \times L$ denoting datatype property assertions that connect instances to their literal values. In our method, Slither compiles the contract to extract its AST and decomposes statements into operations to build IR. Slither IR is further converted into a static single assignment (SSA) form by assigning each variable a unique version and adding ϕ-functions to handle control-flow divergence in variable values, making data and control flows explicit for richer analysis. By traversing the Slither IR, we can extract not only contract-level elements as nodes and relations as properties for the instance layer, but also more complex relationships, including contract or function *Inheritance*, function or modifier *Invocation*, and statement-level execution flows that are often difficult to obtain directly from parsing the ASTs.

3.2 Prompt Construction

We design a two-round chain-of-thought (CoT) prompting scheme [44] to translate natural-language defect patterns into SPARQL queries over the contract knowledge graph. As illustrated in Figure 3, each round follows a unified template consisting of Task, Input, Instructions, Output, and Example. The first round guides the model to produce structured reasoning from the natural-language description, while the second round converts this reasoning into an executable SPARQL query. Compared to conventional NL2GQL prompts, our approach refines the Input and Instructions components to align query generation with the contract knowledge graph.

3.2.1 Input. The prompt incorporates a structured CKG schema, enabling the model to interpret domain-specific terminology and relation mappings, as well as a summary of the KG in tabular form, enabling feasibility checks and constraining query generation to the CKG's boundaries. The KG summary is extracted through a pruning strategy that extracts only access-control-related subgraphs to avoid limitations in the LLM context, as described in Algorithm 1.

3.2.2 Instructions. The prompt instruction forms the core of the method, guiding the LLM through a structured CoT process for NL2GQL generation. We decompose this process into seven key steps: Intent Parsing, Slot Justification, KG Feasibility Check, Query Plan Construction, SPARQL Generation, Confidence and Ambiguity

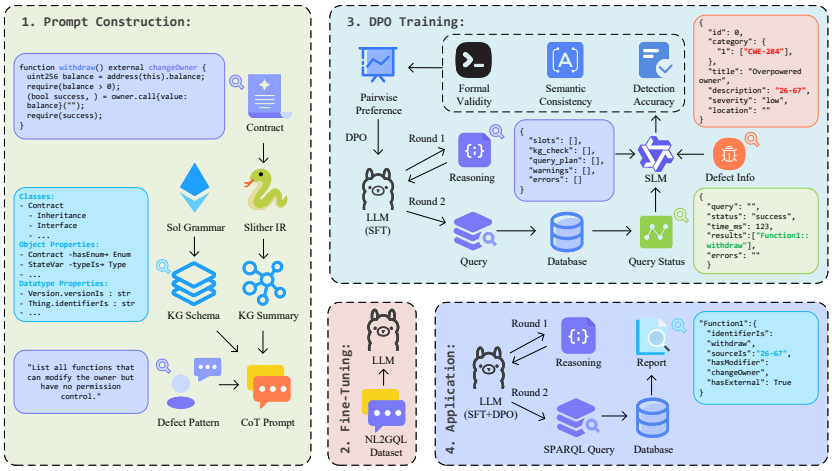


Figure 2: The overall workflow of CKG-LLM, consists of four stages: (1) construction of the contract knowledge graph and prompt design; (2) SFT for guiding LLMs in NL2GQL tasks; (3) DPO-based training to enhance the effectiveness of GQL in detecting vulnerabilities; and (4) LLM-assisted query generation for real-world applications. (Icons adapted from Iconfont.)

Handling, and Output Formatting. Intent Parsing normalizes target entities and conditions based on the Contract KG Schema, while Slot Justification maps natural-language cues to corresponding schema fields. The KG Feasibility Check verifies whether slot values align with ontology or instance elements and flags mismatches. Query Plan Construction identifies the minimal triple patterns and constraints, which SPARQL Generation then converts into ontology-consistent queries. Confidence and Ambiguity Handling categorizes the model's outputs and offers alternatives when ambiguity arises, and Output Formatting standardizes the final query for consistency.

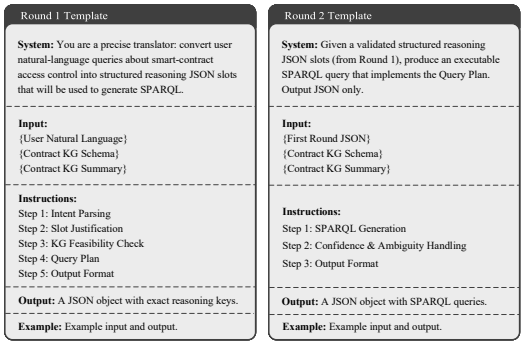


Figure 3: An overview of the two-round prompt template.

3.3 Fine-Tuning

Supervised fine-tuning (SFT) [7] is designed to familiarize the model with GQL paradigms. We choose Llama-3.1-8B-Instruct [23] as the base model due to its code generation capability and open-source nature. Regarding dataset choice, we utilize LC-QuAD 2.0 [5], which comprises 55,782 complex query pairs, aligning well with the query needs of smart contract knowledge graphs. We also adopt the low-rank adaptation (LoRA) method [11] to reduce computational cost and memory usage while preserving model expressiveness.

3.4 DPO Training

To enhance the LLM's understanding of contract syntax and vulnerability detection rules, we propose a reinforcement learning approach that incorporates feedback from a small language model agent (RLAF). We use 1,291 contracts from the FORGE dataset [4] (see Section 4) with simplified access-control–related CWE patterns [24–30] to construct prompts through the module in Stage 1. Which are then fed into the SFT model from Stage 2. Each prompt undergoes a two-round dialogue to generate a SPARQL query, and each prompt is executed twice to form comparison pairs. A Qwen2.5-3B-Instruct agent [35] [22] utilizes a scoring function to evaluate the LLM's reasoning outputs, generated queries, execution results from the database, and defect information from the dataset, and ranks the query pairs for direct preference optimization (DPO) [36] [43]. The evaluation is based on three dimensions: Formal Validity (whether the query executes correctly and its retrieves results correlate to the NL intent), Semantic Consistency (whether the query semantically aligns with the NL intent, respects the schema, and uses compatible filtering conditions), and Detection Accuracy (how effectively the query detects vulnerabilities, measured via localization accuracy, type coverage, and the precision of retrieved instances).

Algorithm 1: Pruning Contract KG for Access Control

Input : Contract knowledge graph $G = (V, E)$
Output: Pruned subgraph $G' \subseteq G$

$R \leftarrow \emptyset$ // Relevant nodes
foreach *function* $f \in V$ **do**
  if *visibility*$(f) \in \{public, external\}$ **then**
    if f *has modifier* $\in \{onlyOwner, onlyRole\}$ *or contains statement involving* $\{msg.sender, require, hasRole\}$ **then**
      $R \leftarrow R \cup \{f\}$
      $R \leftarrow R \cup$ variables $\{v \mid v \in \{\text{owner, roles}\}\}$ used in f
$G' \leftarrow \{(s, p, o) \in E \mid s \in R \vee o \in R\}$
return G'

3.5 Application

In real-world contract detection, the framework takes user-defined NL vulnerability descriptions and smart contract source code as inputs and processes them using the models in Stage 1. It then constructs a two-round domain-enhanced chain-of-thought prompt,

which is passed to the pretrained LLM in Stage 3 to generate a SPARQL query tailored for vulnerability localization. This query is executed on the contract knowledge graph stored in the graph database to retrieve entities and produce the final detection results.

4 Experimental Evaluation

In this section, we will address the following research questions: **RQ1 (Quality):** How is the quality of LLM generated queries? **RQ2 (Accuracy):** Can SPARQL query-based detection accurately identify access control vulnerabilities in real-world smart contracts? **RQ3 (Efficiency):** How efficient is CKG-LLM compared with existing frameworks for access control vulnerability detection?

Our experiments were conducted on a server equipped with an Intel i7-13700KF CPU, 32GB of RAM, and an NVIDIA RTX 4070 GPU, running on Ubuntu 24.04. The experimental dataset was sourced from FORGE [4], from which we filtered 1,844 contracts that contained defect labels according to seven access control-related CWE patterns [24–30]. Among them, 1,291 contracts were utilized for DPO training in Section 3.4, and 553 contracts for evaluation.

RQ1 Quality: To address RQ1, we reused the training pipeline from Stage 3 to construct the evaluation workflow. Both the SFT and the SFT+DPO trained LLMs were employed to assess their effectiveness on the NL2GQL task. Similar to the DPO training setup, the evaluation inputs consisted of 553 contracts with access-control-related defect labels from the FORGE subset [4], along with the defect pattern derived from the concise CWE-284 description [26]. In this experiment, we made a minor modification to the prompting strategy. For each contract, the LLM was asked to execute the prompt once to produce a single query, while the SLM was asked to provide scoring rather than ranking of generated results. The average generation quality was then computed across all 553 contracts, with the score distribution presented in Figure 4. The results in Figure 4 demonstrate that both models achieved relatively high scores in the NL2GQL task, while incorporating DPO into SFT improves the quality of SPARQL generation. The SFT model achieved an average score of 0.693, and the reinforcement-learned models scored even higher, at 0.756. This shift indicates that DPO enhances the model's ability to generate more accurate and reliable queries.

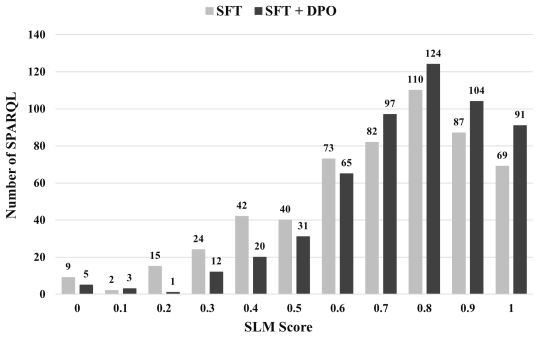


Figure 4: The SLM score distribution of the SFT and SFT+DPO trained model over 553 contracts in NL2GQL tasks.

RQ2 Accuracy: To address RQ2, we assessed the detection accuracy and F1 scores of the Llama base, SFT, and SFT+DPO models across 553 contracts against the CWE-284 pattern [26]. We followed the setup in Stage 4 and compared their query outputs against the dataset labels with the assistance of SLM. The statistical results are summarized in Table 1. As shown in the table, SFT training yields a notable improvement in precision. It reduces recall compared to the baseline, demonstrating the effectiveness of supervised fine-tuning in aligning the model with the NL2GQL task. Furthermore, the DPO training yields consistent gains across all metrics, indicating that preference optimization not only reduces false positives but also enhances the overall robustness of vulnerability detection.

Table 1: Detection accuracy comparison of baseline, SFT, and SFT+DPO models. Metrics: true positives (TP), false positives (FP), false negatives (FN), precision, recall, and F1 score.

Model	#TP	#FP	#FN	Prec(%)	Recall(%)	F1(%)
Base	276	223	180	55.3	60.5	57.8
SFT	314	197	134	61.4	70.1	65.5
DPO+SFT	**348**	**128**	**105**	**73.1**	**76.8**	**74.9**

RQ3 Efficiency: To address RQ3, we measured the total detection time of CKG-LLM, covering the entire pipeline of prompt construction, LLM response, and report generation based on the accuracy validation process in Section 4. For comparison, we further introduced Achecker [8], an access control vulnerability detection framework, and GPTScan [41], an LLM-assisted static analysis framework, to analyze both accuracy and efficiency. The results are presented in Table 2. The comparative results highlight that CKG-LLM offers the most balanced performance in terms of both accuracy and efficiency. Although Achecker achieves the highest F1 score (81.4%), this advantage comes at the cost of higher runtime. GPTScan, on the other hand, performs competitively but falls short in both accuracy and efficiency. In contrast, CKG-LLM achieves a strong F1 score (74.9%) while delivering the lowest overall runtime and the fastest average detection time per contract.

Table 2: Detection accuracy and efficiency comparison of Achecker, GPTScan, and CKG-LLM. Metrics: precision, recall, F1 score, total detection time, and average detection time.

Tool	Prec(%)	Recall(%)	F1(%)	Time(s)	ADT(s)
Achecker	**83.3**	**79.5**	**81.4**	7442	13.5
GPTScan	63.1	66.3	64.7	8754	15.8
CKG-LLM	73.1	76.8	74.9	**6248**	**11.3**

5 Future Plans

In future work, we aim to enhance both the scalability and accuracy of CKG-LLM. Specifically, we will refine the prompt design and pruning strategy to support more vulnerability types, such as integer overflow or reentrancy. Additionally, we plan to incorporate retrieval-augmented generation into Section 3.4 and Section 3.5 by encoding contract knowledge graphs into latent embeddings, thereby reducing context overhead and guiding the LLM to generate queries compliant with Solidity grammar and contract syntax.

Acknowledgments

This work is sponsored by the National Natural Science Foundation of China (No.62402146 & No.62362021). AI-based tools are utilized solely for language polishing during manuscript preparation.

References

[1] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology* 15, 3 (2024), 1–45.

[2] Chong Chen, Jiachi Chen, Lingfeng Bao, David Lo, Yanlin Wang, Zhenyu Shan, Ting Chen, Guangqiang Yin, Jianxing Yu, and Zibin Zheng. 2025. Trace: Securing Smart Contract Repository Against Access Control Vulnerability. *arXiv preprint arXiv:2510.19254* (2025).

[3] Jiachi Chen, Zhenzhe Shao, Shuo Yang, Yiming Shen, Yanlin Wang, Ting Chen, Zhenyu Shan, and Zibin Zheng. 2025. NumScout: Unveiling Numerical Defects in Smart Contracts using LLM-Pruning Symbolic Execution. *IEEE Transactions on Software Engineering* (2025).

[4] Jiachi Chen, Yiming Shen, Jiashuo Zhang, Zihao Li, John Grundy, Zhenzhe Shao, Yanlin Wang, Jiashui Wang, Ting Chen, and Zibin Zheng. 2025. FORGE: An LLM-driven Framework for Large-Scale Smart Contract Vulnerability Dataset Construction. *arXiv preprint arXiv:2506.18795* (2025).

[5] Mohnish Dubey. 2025. A large data set of natural language queries with corresponding SPARQL queries for Wikidata and Dbpedia2018. Website. https://github.com/AskNowQA/LC-QuAD2.0

[6] Josselin Feist, Gustavo Grieco, and Alex Groce. 2019. Slither: a static analysis framework for smart contracts. In *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 8–15.

[7] Heshan Fernando, Han Shen, Parikshit Ram, Yi Zhou, Horst Samulowitz, Nathalie Baracaldo, and Tianyi Chen. 2024. Mitigating forgetting in llm supervised fine-tuning and preference learning. *arXiv preprint arXiv:2410.15483* (2024).

[8] Asem Ghaleb, Julia Rubin, and Karthik Pattabiraman. 2023. Achecker: Statically detecting smart contract access control vulnerabilities. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 945–956.

[9] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).

[10] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, et al. 2021. Knowledge graphs. *ACM Computing Surveys (Csur)* 54, 4 (2021), 1–37.

[11] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.

[12] Tianyuan Hu, Bixin Li, Zhenyu Pan, and Chen Qian. 2023. Detect defects of solidity smart contract based on the knowledge graph. *IEEE Transactions on Reliability* 73, 1 (2023), 186–202.

[13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[14] Wei Li, Yuhong Nan, Mingxi Ye, Jingwen Zhang, Peilin Zheng, and Zibin Zheng. 2025. ASTRO: Detecting Access Control Vulnerabilities in Smart Contracts via Graph Similarity Comparison. *IEEE Transactions on Software Engineering* (2025).

[15] Xiaoqi Li, Ting Chen, Xiapu Luo, and Chenxu Wang. 2021. Clue: towards discovering locked cryptocurrencies in ethereum. In *Proceedings of the 36th Annual ACM Symposium on Applied Computing*. 1584–1587.

[16] Xiaoqi Li, Ting Chen, Xiapu Luo, and Jiangshan Yu. 2020. Characterizing erasable accounts in ethereum. In *International Conference on Information Security*. Springer, 352–371.

[17] Xiaoqi Li, Ting Chen, Xiapu Luo, Tao Zhang, Le Yu, and Zhou Xu. 2020. Stan: Towards describing bytecodes of smart contract. In *2020 IEEE 20th International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 273–284.

[18] Xiaoqi Li, Peng Jiang, Ting Chen, Xiapu Luo, and Qiaoyan Wen. 2020. A survey on the security of blockchain systems. *Future generation computer systems* 107 (2020), 841–853.

[19] Xiaoqi Li, Wenkai Li, Zhijie Liu, Meikang Qiu, Zhiquan Liu, Sen Nie, Zongwei Li, Shi Wu, and Yuqing Zhang. 2025. ScamSweeper: Detecting Illegal Accounts in Web3 Scams via Transactions Analysis. *IEEE Transactions on Information Forensics and Security* Early Access (2025), 1–14.

[20] Xiaoqi Li, Zongwei Li, Wenkai Li, Yuqing Zhang, and Xin Wang. 2025. No More Hidden Pitfalls? Exposing Smart Contract Bad Practices with LLM-Powered Hybrid Analysis. *ACM Transactions on Software Engineering and Methodology* (2025).

[21] Yuanyuan Liang, Keren Tan, Tingyu Xie, Wenbiao Tao, Siyuan Wang, Yunshi Lan, and Weining Qian. 2024. Aligning large language models to a domain-specific graph database for nl2gql. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 1367–1377.

[22] Tula Masterman, Sandi Besen, Mason Sawtell, and Alex Chao. 2024. The landscape of emerging ai agent architectures for reasoning, planning, and tool calling: A survey. *arXiv preprint arXiv:2404.11584* (2024).

[23] Meta. 2025. Introducing Llama 3.1: Our most capable models to date. Website. https://ai.meta.com/blog/meta-llama-3-1/

[24] MITRE. 2025. CWE-269: Improper Privilege Management. Website. https://cwe.mitre.org/data/definitions/269.html

[25] MITRE. 2025. CWE-276: Incorrect Default Permissions. Website. https://cwe.mitre.org/data/definitions/276.html

[26] MITRE. 2025. CWE-284: Improper Access Control. Website. https://cwe.mitre.org/data/definitions/284.html

[27] MITRE. 2025. CWE-285: Improper Authorization. Website. https://cwe.mitre.org/data/definitions/285.html

[28] MITRE. 2025. CWE-732: Incorrect Permission Assignment for Critical Resource. Website. https://cwe.mitre.org/data/definitions/732.html

[29] MITRE. 2025. CWE-862: Missing Authorization. Website. https://cwe.mitre.org/data/definitions/862.html

[30] MITRE. 2025. CWE-863: Incorrect Authorization. Website. https://cwe.mitre.org/data/definitions/863.html

[31] Iulian Neamtiu, Jeffrey S Foster, and Michael Hicks. 2005. Understanding source code evolution using abstract syntax tree matching. In *Proceedings of the 2005 international workshop on Mining software repositories*. 1–5.

[32] OWASP. 2025. OWASP Top Ten. Website. https://owasp.org/Top10/2025/0x00_2025-Introduction/

[33] Hongli Peng, Wenkai Li, and Xiaoqi Li. 2025. Mining characteristics of vulnerable smart contracts across lifecycle stages. *IET Blockchain* 5, 1 (2025), e70016.

[34] Jorge Pérez, Marcelo Arenas, and Claudio Gutierrez. 2009. Semantics and complexity of SPARQL. *ACM Transactions on Database Systems (TODS)* 34, 3 (2009), 1–45.

[35] Qwen. 2025. Alibaba Cloud's general-purpose AI models. Website. https://huggingface.co/Qwen/Qwen2.5-3B-Instruct

[36] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741.

[37] Yiming Shen, Kunhua Li, Lin Mao, Wenkai Li, and Xiaoqi Li. 2023. IntelliCon: Confidence-Based Approach for Fine-Grained Vulnerability Analysis in Smart Contracts. In *International Conference on Blockchain and Trustworthy Systems*. Springer, 45–59.

[38] Chaofan Shou, Jing Liu, Doudou Lu, and Koushik Sen. 2024. Llm4fuzz: Guided fuzzing of smart contracts with large language models. *arXiv preprint arXiv:2401.11108* (2024).

[39] Solidity. 2025. Language Grammar. Website. https://docs.soliditylang.org/en/latest/grammar.html

[40] Solidity. 2025. Solidity Language Grammar. Website. https://github.com/solidity-parser/antlr

[41] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. 2024. Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[42] Nick Szabo. 1996. Smart contracts: building blocks for digital markets. *EXTROPY: The Journal of Transhumanist Thought,(16)* 18, 2 (1996), 28.

[43] Zhichao Wang, Bin Bi, Shiva Kumar Pentyala, Kiran Ramnath, Sougata Chaudhuri, Shubham Mehrotra, Xiang-Bo Mao, Sitaram Asur, et al. 2024. A comprehensive survey of llm alignment techniques: Rlhf, rlaif, ppo, dpo and more. *arXiv preprint arXiv:2407.16216* (2024).

[44] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[45] Zhiyuan Wei, Jing Sun, Zijiang Zhang, Xianhao Zhang, Meng Li, and Zhe Hou. 2024. Llm-smartaudit: Advanced smart contract vulnerability detection. *arXiv preprint arXiv:2410.09381* (2024).

[46] Shihao Xia, Mengting He, Shuai Shao, Tingting Yu, Yiying Zhang, and Linhai Song. 2025. SymGPT: Auditing Smart Contracts via Combining Symbolic Execution with Large Language Models. *arXiv preprint arXiv:2502.07644* (2025).

[47] Hanxiang Xu, Wei Ma, Ting Zhou, Yanjie Zhao, Kai Chen, Qiang Hu, Yang Liu, and Haoyu Wang. 2025. CKGFuzzer: LLM-Based Fuzz Driver Generation Enhanced By Code Knowledge Graph. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 243–254.

[48] Shenhui Zhang, Wenkai Li, Xiaoqi Li, and Boyi Liu. 2022. Authros: Secure data sharing among robot operating systems based on ethereum. In *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 147–156.

[49] Yuhang Zhou, Yu He, Siyu Tian, Yuchen Ni, Zhangyue Yin, Xiang Liu, Chuanjun Ji, Sen Liu, Xipeng Qiu, Guangnan Ye, et al. 2023. R^3-NL2GQL: A Model Coordination and Knowledge Graph Alignment Approach for NL2GQL. *arXiv preprint arXiv:2311.01862* (2023).